Rural-First POS, Inventory & Barcode Billing System (V1) — End-to-End Product Blueprint, 21-Day Execution Plan & Coding LLM Master Prompts

1. Executive Vision & Product Positioning

This blueprint defines the complete architecture, feature specification, 21-day build schedule, and copy-paste coding LLM prompts to build a **laptop-first, 100% offline-capable Point of Sale (POS), Inventory, and Barcode Billing System** modeled on core workflows from myBillBook and tailored specifically for **Indian electronics shops and rural/semi-urban retail stores**.

Why This Wins in Your Electronics Shop First — and Rural Markets Next

Commercial billing apps often struggle in rural and semi-urban towns because they assume uninterrupted broadband, high digital literacy, expensive dedicated thermal label printers, and recurring annual subscription readiness. Your product solves these exact bottlenecks:

Dimension	**Standard Urban POS (myBillBook)**	**Your Rural-First Electronics & Retail POS (V1)**
Internet Dependency	Cloud-first with desktop sync (myBillBook PC Billing)	**100% Offline-First on Laptop** using Dexie.js IndexedDB; zero internet required to bill, scan, or print
Unbarcoded / Loose Items	Barcode generation & label printing (myBillBook Barcode Generator)	**1-Click Free Barcode Generator** using JsBarcode + **A4 Ordinary Printer Sticker Grid (40/65 labels per page)** so shops don't need a separate label printer
Electronics Shop Specifics	Category stock, serial/IMEI fields, warranty & past invoice lookup (myBillBook Business Type Guide)	Built-in **IMEI / Serial Number capture**, **Warranty Period tracker (6/12/24 months)**, and **Replacement/Repair lookup by Customer Phone**

Dimension	Standard Urban POS (myBillBook)	Your Rural-First Electronics & Retail POS (V1)
Customer Bill Sharing	WhatsApp, SMS & PDF sharing (myBillBook Retail Billing)	**Zero-Cost WhatsApp Click-to-Chat Sharing** via WhatsApp Official `wa.me` Links + Thermal (58mm/80mm) & A4 Print
Customer Payment Flexibility	Cash, UPI, Card, Credit/Ledger (myBillBook POS Features)	**Dynamic Bill-Specific UPI QR Code** per NPCI UPI Linking Specifications v1.6, **Cash + UPI Split Payment**, and **1-Click Udhaar (Khata) Ledger**
Invoice Mode	GST & Non-GST Bills (myBillBook Home)	Instant toggle between **Kachha Bill / Estimate (Non-GST)** and **Pakka Tax Invoice (CGST/SGST)** on the billing screen

Suggested Product Names (For Future Branding)

- **GramBill / GramPOS** — Emphasizes rural reach (*Gram*) and fast billing.
- **DukaanMitra POS** — Warm, trustworthy local positioning ("Shopkeeper's Friend").
- **TezKhata & Bill** — Highlights speed (*Tez*) and integrated credit/billing (*Khata + Bill*).
- **BharatBill Pro** — Scales naturally from your electronics shop to multi-category stores across towns and villages.

2. Architecture & Technology Stack (Laptop V1 -> Mobile/Web V2 Ready)

To ship a rock-solid V1 on your laptop within **15 to 21 days** while ensuring you **do not have to rewrite the code** when expanding to mobile apps and cloud multi-shop SaaS later, use a **Local-First Progressive Web Application (PWA)** architecture.

- **Frontend & Desktop Runtime**: React 18 + TypeScript powered by Vite and styled with Tailwind CSS. Installing the app as a desktop PWA via Chrome or Edge on Windows/Linux/macOS gives you a native desktop icon, full-screen window, offline asset caching via Service Workers, and zero installer complexity.
- **Local Offline Database (V1 Core)**: Dexie.js (a transactional wrapper around browser IndexedDB). It stores tens of thousands of products, barcodes, serial numbers, and

invoices directly on the laptop hard drive with sub-millisecond query speed and zero cloud cost.

- **Barcode Generation Library**: JsBarcode to generate standard `CODE128` and `EAN13` barcodes in SVG/Canvas for unbarcoded items and print them onto A4 sticker sheets or thermal labels.
- **Barcode Scanning Engine (Dual Mode)**:
    1. **Primary (Hardware USB / Wireless 2D Scanner)**: Global JavaScript `keydown` buffer listener that detects rapid HID keyboard-wedge input (`< 40ms` between characters ending in `Enter`) so you can scan items anywhere on the POS screen without clicking an input box first.
    2. **Secondary (Laptop Webcam / Future Mobile Camera)**: html5-qrcode for camera-based barcode and QR scanning when a hardware scanner is unavailable.
- **Dynamic UPI QR Code Generator**: node-qrcode encoding an exact NPCI UPI Linking URI (`upi://pay?pa=...&am=...`) so customers scan once and see the exact bill total pre-filled in Google Pay, PhonePe, or Paytm (NPCI UPI).
- **Zero-Cost WhatsApp Bill Delivery**: WhatsApp Click-to-Chat (`wa.me`) pre-populated with a structured bilingual receipt and UPI payment link, plus automatic PDF clipboard/download support.
- **Phase 2 Cloud & Multi-Shop SaaS Backend (Future Expansion)**: Supabase (PostgreSQL + Auth + Row-Level Security). Every local table in V1 already includes `shop_id`, `updated_at`, and `sync_status` fields so adding cloud sync and mobile companion apps later requires only plugging in a background sync worker.

3. Complete Database Schema (Local-First + Multi-Tenant Ready)

Every table includes `id` (UUID), `shop_id` (defaults to `'local-shop-01'` in V1), `created_at`, `updated_at`, and `sync_status` (`'synced' | 'pending'`).

Table 1: `shop_settings`

- `id`: string (Primary Key)
- `shop_name`: string (e.g., `"Your Electronics & Electricals"`)
- `owner_name`: string
- `phone`: string (10-digit mobile number)
- `address`: string (Village/Town, District, State, PIN)
- `gstin`: string (Optional — if blank, defaults to Non-GST / Cash Memo mode)
- `upi_id`: string (Merchant VPA, e.g., `yourshop@okaxis` or `9876543210@ybl`)
- `upi_payee_name`: string
- `invoice_prefix`: string (e.g., `"INV-2026-"`)
- `next_invoice_number`: number (Auto-incrementing counter)
- `default_print_size`: `'thermal_80mm' | 'thermal_58mm' | 'a4' | 'a5'`

- `language_mode`: `'en' | 'hi' | 'hinglish'`
- `terms_and_conditions`: string (e.g., `"1. Goods once sold will not be taken back. 2. Warranty as per manufacturer policy."`)

Table 2: `products`

- `id`: string (UUID Primary Key)
- `sku_code`: string (Unique internal code, e.g., `"ELEC-1001"`)
- `barcode`: string (Indexed — manufacturer EAN/UPC barcode OR system-generated `CODE128` barcode)
- `is_custom_barcode`: boolean (`true` if generated by our system for an unbarcoded item)
- `name`: string (e.g., `"9W LED Bulb Cool Day Light"`)
- `name_local`: string (Optional Hindi/local name, e.g., `"9W एलईडी बल्ब"`)
- `category`: string (e.g., `"Lighting"`, `"Cables & Wires"`, `"Mobile Accessories"`, `"Appliances"`, `"Spare Parts"`)
- `brand`: string (e.g., `"Havells"`, `"Philips"`, `"Local / Generic"`)
- `unit`: `'pcs' | 'meter' | 'box' | 'pair' | 'set' | 'kg'`
- `purchase_price`: number (Cost price for profit calculation — hidden during customer checkout)
- `mrp`: number (Maximum Retail Price printed on box/label)
- `selling_price`: number (Default counter selling price)
- `min_selling_price`: number (Floor price during rural bargaining so staff never sell at a loss)
- `gst_rate`: number (`0 | 5 | 12 | 18 | 28`)
- `tax_inclusive`: boolean (Default `true` for retail counter simplicity)
- `hsn_code`: string (Optional HSN code for GST bills)
- `stock_qty`: number (Real-time available quantity)
- `low_stock_threshold`: number (Default `5` — triggers low-stock badge)
- `track_serial_imei`: boolean (`true` for phones, inverters, batteries, TVs, ceiling fans with serial warranty)
- `default_warranty_months`: number (e.g., `0`, `6`, `12`, `24`, `36`)
- `rack_location`: string (e.g., `"Rack B - Shelf 2"` for quick physical lookup)

Table 3: `customers`

- `id`: string (UUID Primary Key)
- `name`: string
- `phone`: string (Unique Indexed — 10-digit WhatsApp/mobile number)
- `village_or_area`: string (Important in rural shops where multiple customers share the same first name)
- `gstin`: string (Optional for B2B sub-dealer sales)
- `current_udhaar_balance`: number (Positive = customer owes the shop money)

- `credit_limit`: number (Default `5000` — warns shopkeeper if unpaid balance exceeds limit)
- `total_purchases`: number
- `last_visit_date`: string (ISO timestamp)

Table 4: `invoices`

- `id`: string (UUID Primary Key)
- `invoice_number`: string (Unique Indexed, e.g., `"INV-2026-0001"`)
- `invoice_type`: `'gst_tax_invoice' | 'non_gst_estimate'`
- `customer_id`: string | null (Null for anonymous walk-in cash customers)
- `customer_name`: string (Snapshot on bill)
- `customer_phone`: string (Snapshot on bill)
- `customer_village`: string
- `subtotal`: number
- `item_discount_total`: number
- `extra_bill_discount`: number (Bargaining / round-off discount at counter)
- `taxable_amount`: number
- `cgst_total`: number
- `sgst_total`: number
- `round_off`: number
- `grand_total`: number (Final payable amount in INR)
- `total_cost_price`: number (Internal snapshot to compute exact daily net profit)
- `payment_status`: `'paid' | 'partial_udhaar' | 'full_udhaar'`
- `payment_mode`: `'cash' | 'upi' | 'card' | 'split' | 'udhaar'`
- `cash_paid`: number
- `upi_paid`: number
- `card_paid`: number
- `udhaar_amount`: number (`grand_total - (cash_paid + upi_paid + card_paid)`)
- `upi_reference_note`: string (Optional last 4 digits of UTR)
- `notes`: string
- `status`: `'completed' | 'cancelled' | 'returned'`
- `created_at`: string (ISO timestamp)

Table 5: `invoice_items`

- `id`: string (UUID Primary Key)
- `invoice_id`: string (Foreign Key -> `invoices.id`)
- `product_id`: string (Foreign Key -> `products.id`)
- `product_name`: string
- `barcode`: string
- `quantity`: number

- `unit`: string
- `mrp`: number
- `unit_price`: number
- `discount_amount`: number
- `gst_rate`: number
- `tax_amount`: number
- `line_total`: number
- `serial_or_imei_numbers`: string[] (Array of scanned/entered serial numbers for warranty tracking)
- `warranty_months`: number
- `warranty_expiry_date`: string | null (Auto-calculated from `created_at + warranty_months`)

Table 6: `khata_transactions` (Customer Credit / Udhaar Ledger)

- `id`: string (UUID Primary Key)
- `customer_id`: string (Foreign Key -> `customers.id`)
- `invoice_id`: string | null (Linked invoice if credit was taken during a sale)
- `type`: `'credit_given' | 'payment_received'`
- `amount`: number
- `payment_mode`: `'cash' | 'upi' | 'none'`
- `balance_after`: number
- `remarks`: string
- `created_at`: string

Table 7: `stock_movements`

- `id`: string (UUID Primary Key)
- `product_id`: string (Foreign Key -> `products.id`)
- `change_type`: `'opening_stock' | 'purchase_in' | 'sale_out' | 'return_in' | 'damage_adjustment'`
- `qty_change`: number (`+` for stock in, `-` for stock out)
- `reference_id`: string (Invoice ID or Supplier Purchase Note)
- `created_at`: string

4. Deep-Dive Functional Specifications (Every Minor Detail)

4.1 Free Barcode Generation & Sticker Printing Engine (For Unbarcoded Rural/Electronics Items)

In an electronics or rural shop, 30%–50% of items (local extension boards, loose copper wire per meter, unbranded mobile chargers, capacitor/resistor packs, repair spares) lack factory barcodes (myBillBook Barcode Billing Guide).

1. **Auto-Generation Logic**:
    - When adding or editing a product, if the `Barcode` field is left empty or the user clicks **"Generate Free Barcode"**, the system auto-generates a unique 10-character alphanumeric code using the format `SKU` + 7-digit zero-padded counter (e.g., `SKU0001042`) and sets `is_custom_barcode = true`.
    - Renders a crisp `CODE128` barcode SVG immediately using JsBarcode (`format: "CODE128"`, `width: 1.8`, `height: 42`, `displayValue: true`).
2. **Bulk & Single Barcode Sticker Print Studio**:
    - Dedicated screen where the shopkeeper selects one or more products and enters the number of stickers needed (defaulting to current `stock_qty` or custom count like `10`, `25`, `65`).
    - **Paper Format 1 — Standard A4 Sticker Sheet (38mm × 21.2mm, 65 labels per A4 sheet — 5 columns × 13 rows)** OR **(48.5mm × 25.4mm, 40 labels per A4 sheet — 4 columns × 10 rows)**: Works on any basic ₹8,000 inkjet/laser printer using ₹1-per-sheet self-adhesive A4 sticker paper. Includes a **"Skip Used Stickers (Start from Row X, Column Y)"** setting so partially used A4 sticker sheets are never wasted!
    - **Paper Format 2 — Thermal Barcode Label Roll (50mm × 25mm or 38mm × 25mm, 1 or 2 labels per row)**: For shops with thermal label printers.
    - **Each Sticker Prints**: Shop Name (top bold, 7pt), Product Name (truncated cleanly at 22 chars, 8pt), Barcode SVG (`CODE128`), Barcode Text, and **MRP / Selling Price (₹XXX)** in bold at the bottom.

4.2 High-Speed Laptop POS Billing Screen (Under 8 Seconds per Bill)

Modeled after myBillBook Windows POS, the POS screen is designed for speed during peak shop hours:

1. **Global Hardware Barcode Scanner Listener**:
    - External USB/wireless barcode scanners act as rapid keyboards. Implement a global `window.addEventListener('keydown', ...)` hook that measures time between keystrokes.
    - If characters arrive within `< 40ms` of each other and end with `Enter`, capture the string as a barcode scan even if the cursor is not inside the search bar.

- On match: Immediately add `+1` quantity of that product to the active cart and play a short high-pitched **Web Audio API success beep (`880Hz, 120ms`)**.
- On unknown barcode: Play a double low-pitched **error buzz (`220Hz, 250ms`)** and open a **"Quick Add New Product for Scanned Barcode"** modal with the scanned barcode pre-filled!

2. **Electronics Serial / IMEI & Warranty Prompt**:
   - If a scanned/selected product has `track_serial_imei = true` (e.g., Mobile Phone, Inverter, Battery, Smart TV, Ceiling Fan), a compact inline input appears under that cart row prompting the shopkeeper to **scan or type the Serial/IMEI number** and auto-fills `Warranty: 12 Months (Expires: DD-MMM-YYYY)` on the printed & digital invoice.
3. **Bargaining & Floor-Price Guardrail**:
   - Rural customers frequently bargain (*"Bhaiya ₹520 ka bill hai, ₹500 le lo"*).
   - Provide a 1-click **"Round Off / Bargain Discount"** field at the bottom of the bill.
   - If the discounted unit price drops below `min_selling_price` or `purchase_price`, show a subtle amber warning icon (`⚠️ Below Floor Price`) so the owner or staff knows their margin before confirming.
4. **Multi-Tab Hold / Park Bill (`F4`)**:
   - Support up to 5 simultaneous open bill tabs (`Bill #1`, `Bill #2`, etc.) so if a customer goes back to pick another wire or bulb, the shopkeeper can bill the next customer in line immediately.
5. **Complete Keyboard Shortcut Bar**:
   - `F1`: Focus Item Search | `F2`: Toggle Camera Scanner | `F3`: Select/Add Customer | `F4`: Hold/Switch Bill Tab | `F6`: Toggle Pakka GST vs. Kachha Estimate Bill | `F8`: Focus Extra Discount | `F9`: Instant Cash Checkout | `F10`: Open Dynamic UPI QR Checkout | `F12`: Save & Print Bill.

4.3 Multi-Mode Customer Payment & Dynamic UPI QR Engine

1. **Dynamic Amount-Locked UPI QR Code**:
   - When the customer chooses **UPI** (or **Split Cash + UPI**), the app constructs an NPCI UPI Linking URI: `upi://pay?pa=${encodeURIComponent(shop.upi_id)}&pn=${encodeURIComponent(shop.upi_payee_name)}&am=${upiAmount.toFixed(2)}&cu=INR&tn=${encodeURIComponent(invoiceNumber)}`
   - Renders a large, high-contrast QR code on the laptop screen using node-qrcode and also embeds a compact QR code at the bottom of the printed/PDF invoice.
   - Why shopkeepers love this: The customer cannot accidentally type `₹50` instead of `₹500` in their UPI app—the exact payable amount is locked into the QR code.
2. **Cash Change Calculator**:
   - When paying by Cash, entering `Cash Received` (e.g., `₹500` note for a `₹340` bill) displays **`Return to Customer: ₹160`** in large green text.

3. **Split Payment (`Cash + UPI`)**:
    - Rural customers often pay partial cash and partial UPI (e.g., `₹200 Cash + ₹350 UPI`). Entering `200` in Cash automatically sets the Dynamic UPI QR code to `₹350.00`.
4. **Udhaar / Khata (Full or Partial Credit)**:
    - If a customer pays `₹300` on a `₹500` bill and says *"Baaki ₹200 agle hafte dunga"*, selecting **Partial Udhaar** requires Customer Name + Phone + Village, records `₹200` in `khata_transactions`, updates `customers.current_udhaar_balance`, and prints **`Previous Due + Current Bill - Paid = Total Balance Due`** right on the receipt!

4.4 Digital Bill Sharing (WhatsApp) & Dual Printing (Thermal + A4)

1. **Instant Zero-Cost WhatsApp Receipt (WhatsApp Click-to-Chat)**:
    - Clicking **"Share on WhatsApp"** opens `https://wa.me/91${customerPhone}?text=${encodedReceiptText}`.
    - The message includes the shop header, invoice number, date, itemized list with warranty expiry dates, grand total, payment status, remaining Udhaar balance (if any), and a clickable UPI payment link if unpaid.
    - Also includes a **"Download / Copy Bill Image & PDF"** button so the shopkeeper can drag-and-drop the visual invoice into WhatsApp Web in 2 seconds.
2. **Multi-Format Print Engine (`@media print`)**:
    - **80mm / 58mm Thermal Receipt Layout**: Clean monospace/condensed layout with Shop Name, Phone, Invoice #, Date, Customer details, Item table, GST breakdown (if enabled), Grand Total, Warranty notes, and Dynamic UPI QR code.
    - **A4 / A5 Full Invoice Layout**: Professional bordered tax invoice / cash memo suitable for high-value electronics (TVs, Refrigerators, Inverters, Bulk Wiring orders) with signature block and warranty terms.

5. 21-Day (3-Week) End-to-End Execution & Dogfooding Roadmap

Sprint & Days	Milestone Focus	Concrete Deliverables & Verification Criteria
Phase 1: Days 1–4	**Foundation, Offline DB & Shop Setup**	Scaffold Vite + React + TypeScript + Tailwind CSS + Dexie.js PWA. Implement all 7 database tables, seed sample electronics shop data (50 items

Sprint & Days	Milestone Focus	Concrete Deliverables & Verification Criteria
		across LED bulbs, wires, chargers, fans, inverters), and build the **Shop Settings & Backup/Restore** module.
Phase 2: Days 5–9	**Inventory, Serial/IMEI & Free Barcode Studio**	Build Product CRUD with Floor Price, Low-Stock filters, Excel/CSV bulk import, automatic `CODE128` generation via JsBarcode, and the **A4 Sticker Sheet (65/40-up with row/col offset) + Thermal Label Print Studio**.
Phase 3: Days 10–15	**8-Second POS Counter, Payments & Printing**	Build the POS Billing Counter with global USB barcode scanner listener, html5-qrcode webcam fallback, Serial/IMEI warranty modal, Hold Bill tabs, node-qrcode Dynamic UPI QR modal (NPCI Spec), Thermal/A4 print templates, and WhatsApp Click-to-Chat sharing.
Phase 4: Days 16–21	**Khata Ledger, Daily Reports & Live Shop Launch**	Build the **Customer Udhaar (Khata) Ledger** with 1-click WhatsApp reminders, **Warranty Lookup by Customer Phone/Serial**, Daily Cash/UPI/Profit Dashboard, Bilingual (English/Hinglish) toggle, and deploy live in your electronics shop for real counter billing.

6. Copy-Paste Master Prompts for Coding LLMs

Use the following **4 sequential, self-contained prompts** in your coding LLM (Claude, ChatGPT, Cursor, or Windsurf). Feed **Prompt 1** first, verify the app runs, and then feed

Prompts 2, 3, and 4 in order. Everything is specified down to exact file names, schemas, and edge cases so the LLM leaves zero placeholders.

PROMPT 1 of 4: Project Scaffolding, Offline Database (`Dexie.js`), Seed Data & App Shell

````
```markdown
You are a Principal Full-Stack Engineer building "GramBill POS" — an
offline-first, laptop-optimized Point of Sale (POS), Inventory,
Barcode Generator, and Udhaar (Khata) Billing web application designed
for Indian electronics shops and rural retail businesses.

### OBJECTIVE FOR STEP 1
Scaffold the complete React + TypeScript + Tailwind CSS + Dexie.js
(IndexedDB) application architecture, database schema, realistic
Indian electronics shop seed data, global audio feedback utility,
bilingual dictionary (English + Hinglish), and the main responsive
desktop/laptop navigation shell with Shop Settings and 1-Click JSON
Local Backup/Restore.

### TECH STACK & PACKAGES TO USE
- Framework: React 18 + TypeScript + Vite
- Styling: Tailwind CSS + `lucide-react` icons
- Local Offline Database: `dexie` and `dexie-react-hooks`
- Barcode & QR Libraries (install now for upcoming modules):
`jsbarcode`, `qrcode`, `@types/qrcode`, `html5-qrcode`
- Routing/State: Clean tab-based SPA router with keyboard shortcut
listeners

### EXACT FILE STRUCTURE TO GENERATE
1. `package.json` — Include all required dependencies (`react`,
`react-dom`, `dexie`, `dexie-react-hooks`, `lucide-react`,
`jsbarcode`, `qrcode`, `html5-qrcode`, `tailwindcss`).
2. `src/types/pos.ts` — Complete TypeScript interfaces for all 7
database tables:
   - `ShopSettings`: `id`, `shopName`, `ownerName`, `phone`,
`address`, `gstin`, `upiId`, `upiPayeeName`, `invoicePrefix`,
`nextInvoiceNumber`, `defaultPrintSize` (`'thermal_80mm' |
'thermal_58mm' | 'a4'`), `languageMode` (`'en' | 'hinglish'`),
`termsAndConditions`.
   - `Product`: `id`, `skuCode`, `barcode`, `isCustomBarcode`, `name`,
`nameLocal`, `category`, `brand`, `unit` (`'pcs' | 'meter' | 'box' |
'pair' | 'set'`), `purchasePrice`, `mrp`, `sellingPrice`,
````
````

`minSellingPrice`, `gstRate` (`0 | 5 | 12 | 18 | 28`), `taxInclusive`, `hsnCode`, `stockQty`, `lowStockThreshold`, `trackSerialImei`, `defaultWarrantyMonths`, `rackLocation`, `updatedAt`.
  - `Customer`: `id`, `name`, `phone`, `villageOrArea`, `gstin`, `currentUdhaarBalance`, `creditLimit`, `totalPurchases`, `lastVisitDate`.
  - `InvoiceItem`: `productId`, `productName`, `barcode`, `quantity`, `unit`, `mrp`, `unitPrice`, `discountAmount`, `gstRate`, `taxAmount`, `lineTotal`, `serialOrImeiNumbers` (`string[]`), `warrantyMonths`, `warrantyExpiryDate`.
  - `Invoice`: `id`, `invoiceNumber`, `invoiceType` (`'gst_tax_invoice' | 'non_gst_estimate'`), `customerId`, `customerName`, `customerPhone`, `customerVillage`, `items` (`InvoiceItem[]`), `subtotal`, `itemDiscountTotal`, `extraBillDiscount`, `taxableAmount`, `cgstTotal`, `sgstTotal`, `roundOff`, `grandTotal`, `totalCostPrice`, `paymentStatus` (`'paid' | 'partial_udhaar' | 'full_udhaar'`), `paymentMode` (`'cash' | 'upi' | 'card' | 'split' | 'udhaar'`), `cashPaid`, `upiPaid`, `cardPaid`, `udhaarAmount`, `upiReferenceNote`, `notes`, `status` (`'completed' | 'cancelled'`), `createdAt`.
  - `KhataTransaction`: `id`, `customerId`, `invoiceId`, `type` (`'credit_given' | 'payment_received'`), `amount`, `paymentMode` (`'cash' | 'upi' | 'none'`), `balanceAfter`, `remarks`, `createdAt`.
  - `StockMovement`: `id`, `productId`, `changeType` (`'opening_stock' | 'purchase_in' | 'sale_out' | 'return_in' | 'adjustment'`), `qtyChange`, `referenceId`, `createdAt`.
3. `src/db/database.ts` — Dexie subclass `GramBillDB` defining tables `shopSettings`, `products`, `customers`, `invoices`, `khataTransactions`, and `stockMovements` with proper indexes (`&id, barcode, skuCode, name, category` on `products`; `&id, &phone, name, villageOrArea` on `customers`; `&id, &invoiceNumber, customerId, customerPhone, createdAt` on `invoices`).
  - Include an automatic `seedInitialData()` function that populates:
    a) Default `ShopSettings` for an Indian Electronics & Electricals shop (with sample UPI ID `shopowner@upi` so QR generation works out of the box).
    b) 15 realistic electronics shop products (some with manufacturer EAN-13 barcodes like Havells 9W LED Bulb, Anchor 4-Way Extension Board, Polycab 1.5sqmm Wire 90m Coil, Luminous 150Ah Inverter Battery with `trackSerialImei: true` and `defaultWarrantyMonths: 36`, Samsung 25W Type-C Charger with `trackSerialImei: true` and `defaultWarrantyMonths: 12`; and some local/loose items with custom

```
generated barcodes `SKU0001001` like Local 2-Pin Plug Top, Loose PVC Tape, Ceiling Fan Capacitor 2.5mfd, Universal TV Remote).
     c) 4 realistic rural customers with village names and sample Udhaar balances.
4. `src/utils/sound.ts` — Web Audio API synthesizer (zero external MP3 files required) with:
   - `playScanSuccessBeep()`: 880Hz sine wave for 100ms.
   - `playScanErrorBuzz()`: 220Hz sawtooth wave, two pulses of 120ms.
   - `playCheckoutChime()`: Pleasant two-tone C5 -> G5 chime on bill completion.
5. `src/components/Layout.tsx` — High-contrast, laptop-first sidebar + top status bar showing:
   - Offline/Online status badge (`100% Offline Ready — Data Saved on This Laptop`).
   - Quick navigation with keyboard shortcut badges:
     - `[Alt+1]` POS Billing Counter
     - `[Alt+2]` Inventory & Stock
     - `[Alt+3]` Free Barcode & Label Studio
     - `[Alt+4]` Customer Udhaar (Khata)
     - `[Alt+5]` Past Bills & Warranty Lookup
     - `[Alt+6]` Reports & Daily Profit
     - `[Alt+7]` Shop Settings & Backup
   - Language switcher (`English` / `Hinglish`) in the header.
6. `src/components/ShopSettingsView.tsx` — Complete form to edit Shop Name, Phone, Address, GSTIN, Merchant UPI ID (with live UPI QR preview test), Invoice Prefix, Default Print Size (`80mm Thermal`, `58mm Thermal`, `A4`), plus **"Download Full Laptop Backup (.json)"** and **"Restore Backup from File"** buttons.

Write complete, production-ready code for all files above with zero placeholders or `TODO` comments.
```

PROMPT 2 of 4: Inventory Management, Serial/IMEI Tracking & Free Barcode Sticker Studio

```
```markdown
Now implement Step 2 of "GramBill POS": the complete Inventory Management module (`src/components/InventoryView.tsx`) and the Free Barcode & Sticker Label Print Studio (`src/components/BarcodeStudioView.tsx`).
```
```

DETAILED REQUIREMENTS FOR `src/components/InventoryView.tsx`
1. **Top Summary Cards & Filters**:
   - Total SKUs count, Total Stock Value (at Cost & Selling Price), Low Stock Items Alert count (clickable filter), and Unbarcoded/Custom Barcode count.
   - Instant search bar (matches Product Name, Local/Hindi Name, Barcode, SKU, Brand, or Category) + Category filter dropdown + **"Quick Stock-In (+ Purchase)"** button.
2. **Add / Edit Product Modal (Rural & Electronics Optimized)**:
   - Product Name & Optional Hindi/Local Name (`nameLocal`).
   - Category selector (with ability to type a new category) & Brand.
   - **Barcode Field with 1-Click Free Generator**:
     - Can be scanned via USB barcode scanner directly into the input, OR clicked **"Generate Free Barcode"** which auto-assigns the next sequential `SKU000XXXX` code, marks `isCustomBarcode = true`, and renders a live `CODE128` SVG preview using `JsBarcode`.
   - Pricing Row: Purchase Price (`₹`), MRP (`₹`), Counter Selling Price (`₹`), and **Minimum Floor Price (`₹`)** (to prevent staff from bargaining below margin), plus live **Margin % & Profit in ₹** badge.
   - Tax & Unit Row: Unit (`pcs`, `meter`, `box`, `pair`, `set`), GST Rate (`0%`, `5%`, `12%`, `18%`, `28%`), Tax Inclusive toggle (`true` by default), HSN Code.
   - Stock & Location Row: Current Stock Qty, Low Stock Alert Threshold, Rack/Shelf Location (e.g., `"Rack A-2"`).
   - **Electronics Shop Specific Toggle**:
     - Checkbox: `Track Serial / IMEI Number on Sale` (`trackSerialImei`).
     - Number input: `Default Warranty (Months)` (`0`, `3`, `6`, `12`, `18`, `24`, `36`, `60`).
3. **Quick Stock-In (Purchase Entry) Modal**:
   - Allows the shopkeeper to scan or select an existing product, enter newly purchased quantity (`+Qty`), optionally update Purchase/Selling price, and automatically logs a ``'purchase_in'`` entry in `stockMovements`.
4. **CSV Import / Export**:
   - One-click export of all inventory to CSV and sample CSV template download + bulk import.

DETAILED REQUIREMENTS FOR `src/components/BarcodeStudioView.tsx`
Rural shops and electronics stores have many loose or local items without manufacturer barcodes. Build a complete **Free Barcode Generator & Sticker Print Studio**:
1. **Product Selector & Print Queue**:

```
   - Search/filter products (with quick filter for `"Custom Barcodes Only"` or `"All Products"`).
   - Add products to the **Label Print Queue** and specify exact label count per item (with a `"Match Current Stock Qty"` button).
2. **Paper & Printer Layout Selector (Crucial for Rural Shops Without Thermal Label Printers)**:
   - **Layout A — Standard A4 Sticker Sheet (65 Labels per page: 5 cols × 13 rows, 38mm × 21.2mm)**: Works on any regular inkjet/laser printer.
   - **Layout B — Standard A4 Sticker Sheet (40 Labels per page: 4 cols × 10 rows, 48.5mm × 25.4mm)**: Larger text for electronics boxes and cables.
   - **Layout C — Thermal Roll Label (50mm × 25mm, 1-Up or 2-Up continuous)**.
3. **Partial Sheet Saver ("Skip Used Stickers")**:
   - Input field: `Skip first N blank sticker positions (0 to 64)`. If a shopkeeper already printed 12 stickers on an A4 sheet yesterday, entering `12` leaves the first 12 cells empty so the remaining 53 stickers on that same sheet can be used today with zero wastage!
4. **Label Customization & Print Execution**:
   - Checkboxes to toggle: Show Shop Name, Show Product Name, Show MRP (struck through if higher than Selling Price), Show Selling Price (`₹`), Barcode Height slider.
   - Render all labels using `JsBarcode` (`CODE128`) inside a dedicated `@media print` container with exact millimeter CSS grid dimensions (`@page { size: A4; margin: 5mm; }` or thermal size) and a **"Print Barcode Stickers Now"** button (`window.print()`).

Write complete, working TypeScript/React code for both components and wire them into `App.tsx`.
```

PROMPT 3 of 4: 8-Second Laptop POS Billing Counter, Global Scanner Hook, Dynamic UPI QR & Print/WhatsApp Engine

```markdown
Now implement Step 3 of "GramBill POS": the core POS Billing Counter (`src/components/POSCounterView.tsx`), the Global Hardware Barcode Scanner Hook (`src/hooks/useBarcodeScanner.ts`), the Dynamic UPI QR & Multi-Mode Checkout Modal (`src/components/CheckoutModal.tsx`), and the Printable Receipt + WhatsApp Sharing Component (`src/components/InvoiceReceiptModal.tsx`).
```

```
### 1. `src/hooks/useBarcodeScanner.ts` (Hardware USB Wedge + Webcam Support)
- Listen globally to `window` `keydown` events.
- Maintain a character buffer and timestamp of the last keystroke.
- If printable characters arrive with `< 45ms` interval and terminate with `Enter` (and total length >= 4), prevent default form submission and invoke `onScan(scannedCode)`.
- Ensure it works even when the user's cursor is not focused inside an `<input>` box, while ignoring normal slow human typing inside text inputs.

### 2. `src/components/POSCounterView.tsx` (High-Speed Counter Interface)
- **Multi-Tab Hold Bills (`F4`)**: Support up to 5 concurrent bill tabs (`Counter Bill 1`, `Counter Bill 2`, etc.) stored in state/`localStorage` so no active cart is lost on refresh.
- **Invoice Mode Toggle (`F6`)**: Instant switch between:
  - `Kachha Bill / Estimate (Non-GST)` — hides tax columns for simple rural cash memos.
  - `Pakka GST Tax Invoice` — calculates CGST + SGST breakdown per item HSN/GST rate.
- **Left Pane (Product Discovery & Scanner Controls)**:
  - Search input (`F1`) with instant keyboard navigation (`ArrowUp`/`ArrowDown`/`Enter` to add item).
  - Toggle button (`F2`) to open an inline webcam barcode scanner powered by `html5-qrcode`.
  - Quick Category pills (`All`, `Lighting`, `Wires`, `Chargers`, `Batteries`, `Spares`) and clickable product cards showing Stock badge, Rack location, and Selling Price.
  - If a scanned barcode is not found in Dexie `products`, play `playScanErrorBuzz()` and open an inline **"Quick Add Unregistered Item"** modal with the barcode pre-filled so the shopkeeper can price and bill it on the spot!
- **Right Pane (Active Cart, Serial/IMEI Capture & Pricing Guardrails)**:
  - Customer selector bar (`F3`): Search existing customer by Phone/Name/Village or quick-type a new customer's Name, 10-digit WhatsApp Phone, and Village. Displays their existing `Udhaar Balance` badge in red if `> ₹0`.
  - Cart table with Quantity `+/-` (supports decimal quantities like `2.5` meters of wire), editable Unit Price, and Item Discount.
```

```
  - **Floor Price Guardrail**: If the shopkeeper lowers an item's
effective price below `product.minSellingPrice` (or `purchasePrice`),
display a high-visibility amber warning badge (`⚠️ Below Floor Price
₹X`).
  - **Electronics Serial/IMEI & Warranty Row**: For any cart item
where `trackSerialImei === true`, render a compact sub-row right
underneath the item with:
    - Input to scan/type Serial or IMEI number(s) (one per unit
quantity).
    - Editable Warranty Months badge that auto-computes the exact
expiry date (`Warranty till: DD MMM YYYY`).
  - Bottom Bill Summary: Subtotal, Item Savings, **Extra Bargain /
Round-off Discount (`F8`)**, CGST/SGST breakdown, and huge Grand Total
(`₹`).
  - Action Bar: `[F9] Quick Cash Paid`, `[F10] UPI QR / Split / Udhaar
Checkout`.

### 3. `src/components/CheckoutModal.tsx` (Dynamic UPI QR, Cash
Change, Split & Udhaar)
- **Payment Mode Tabs**:
  1. `UPI QR Code`: Generates a real, dynamic NPCI UPI QR code using
`qrcode` encoding:

`upi://pay?pa=${shop.upiId}&pn=${encodeURIComponent(shop.upiPayeeName)
}&am=${payableAmount.toFixed(2)}&cu=INR&tn=${encodeURIComponent(invoic
eNumber)}`
     Displays the QR code large on screen so the customer can scan
directly from the laptop screen with Google Pay, PhonePe, or Paytm,
with the exact bill amount pre-filled!
  2. `Cash`: Includes a **"Cash Received from Customer"** input (with
quick buttons `Exact`, `₹100`, `₹200`, `₹500`, `₹2000`) and displays
**`Return Change to Customer: ₹XX`** in large bold green text.
  3. `Split Payment (Cash + UPI + Udhaar)`: Lets the customer pay
partial Cash (e.g., `₹200`), automatically recalculates the Dynamic
UPI QR code for the remaining amount, or moves any unpaid balance into
**Customer Udhaar (Khata)**.
  4. `Full Udhaar (Credit / Pay Later)`: Requires Customer Name +
10-digit Phone + Village. Warns if the new balance exceeds
`customer.creditLimit`.
- **Atomic Database Commit on Complete**:
  - Within a single Dexie transaction (`db.transaction('rw', ...)`):
    1. Save the `Invoice` record and increment
`shopSettings.nextInvoiceNumber`.
```

```
    2. Deduct `quantity` from `products.stockQty` and append
``'sale_out'`` records to `stockMovements`.
    3. If `udhaarAmount > 0`, update `customers.currentUdhaarBalance`
and insert a ``'credit_given'`` record in `khataTransactions`.
    4. Play `playCheckoutChime()` and transition directly to
`InvoiceReceiptModal`.

### 4. `src/components/InvoiceReceiptModal.tsx` (Thermal 58/80mm + A4
Print + Zero-Cost WhatsApp Share)
- **Format Switcher**: Toggle live preview between `80mm Thermal
Receipt`, `58mm Thermal Receipt`, and `A4 Full Tax Invoice / Cash
Memo`.
- **Receipt Details Included**:
  - Shop Name, Address, Phone, GSTIN (if Pakka bill).
  - Invoice Number, Date/Time, Customer Name, Phone, Village.
  - Itemized table including **Serial/IMEI numbers and Warranty Expiry
Date** under applicable electronics items.
  - Total Savings banner (`"You Saved ₹XX Today!"` — rural customers
love seeing their discount printed).
  - Payment breakdown (`Cash Paid`, `UPI Paid`, `Current Bill Udhaar`,
and `Total Previous + Current Udhaar Balance`).
  - Printed **Dynamic UPI QR Code** at the bottom of the bill if any
balance is unpaid or for instant verification.
- **1-Click Zero-Cost WhatsApp Bill Share**:
  - Formats a clean, itemized receipt text (in English/Hinglish)
including warranty dates, total amount, and UPI payment link, and
opens
`https://wa.me/91${customerPhone}?text=${encodeURIComponent(receiptTex
t)}` in a new tab.
  - Also provide a **"Copy Receipt Text"** button and a **"Print Bill
(`Enter` / `F12`)"** button wired to clean `@media print` CSS that
hides all app chrome and prints only the receipt.

Write complete, production-ready code for all files above.
```
```

## PROMPT 4 of 4: Customer Udhaar (Khata) Ledger, Warranty/Past Bill Lookup & Daily Profit Dashboard

```
```markdown
Now implement the final Step 4 of "GramBill POS":
```

1. `src/components/KhataLedgerView.tsx` — Customer Udhaar (Credit) Management & WhatsApp Payment Recovery.
2. `src/components/InvoicesWarrantyView.tsx` — Past Bills, Sales Returns & Electronics Warranty Lookup by Phone or Serial/IMEI Number.
3. `src/components/ReportsDashboardView.tsx` — Daily Cash/UPI Tally, Net Profit Calculator & Low-Stock Reorder List.

1. `src/components/KhataLedgerView.tsx` (Rural Udhaar Recovery Engine)
- **Top Metrics**: Total Market Udhaar Pending (`₹`), Customers over Credit Limit count, and Today's Udhaar Collected (`₹`).
- **Customer List (Left/Main Table)**:
  - Search by Customer Name, Phone, or **Village/Area filter** (essential for rural shopkeepers who collect dues village-by-village on market days).
  - Sort by Highest Udhaar Balance or Oldest Visit Date.
- **Customer Passbook / Ledger Drawer**:
  - Shows full chronological history of bills taken on credit (`credit_given`) and payments received (`payment_received`).
  - **"Record Payment Received (Jama)" Modal**: Enter amount received via Cash or UPI, automatically deducts from `currentUdhaarBalance`, inserts a ``'payment_received'`` record in `khataTransactions`, and generates a **WhatsApp Payment Receipt + Updated Balance** message.
  - **"Send 1-Click WhatsApp Payment Reminder" Button**:
    - Generates a polite Hinglish message:
      `"Namaste ${customer.name} ji, ${shop.shopName} mein aapka kul baaki Udhaar balance ₹${customer.currentUdhaarBalance} hai. Kripya UPI (${shop.upiId}) ya dukaan par aakar bhugtan karein. Dhanyawad!"`
    - Opens `https://wa.me/91${customer.phone}?text=...` directly.

2. `src/components/InvoicesWarrantyView.tsx` (Electronics Warranty & Past Bill Search)
- **Instant Search Bar**: Search past bills by Invoice Number, Customer Phone, Customer Name, OR **Product Serial / IMEI Number**!
- **Why This Is Critical for Electronics Shops**: When a rural customer brings back a faulty LED bulb, ceiling fan, battery, or charger without the paper slip, the shopkeeper simply types the **Customer's Phone Number** or **scans the Serial/IMEI on the product** and immediately sees:
  - Purchase Date, Invoice Number, Selling Price.
  - **Live Warranty Status Badge**: `✅ UNDER WARRANTY (142 Days Left — Expires 12 Nov 2027)` or `❌ WARRANTY EXPIRED on 04 Aug 2026`.

```
- Ability to view, reprint, WhatsApp-share, or mark a bill as
Cancelled/Returned (which automatically restores stock quantities to
`products.stockQty` and reverses any Udhaar balance).

### 3. `src/components/ReportsDashboardView.tsx` (Owner's Daily Galla
/ Hisaab-Kitaab)
- **Date Range Filter**: `Today`, `Yesterday`, `This Week`, `This
Month`, `Custom Date Range`.
- **Daily Counter Closing ("Galla Millaan") Cards**:
  - **Total Sales Revenue (`₹`)** across `N` bills.
  - **Cash in Drawer (`₹`)** (Cash Sales + Cash Udhaar Collections —
exact cash that should be inside the physical cash box).
  - **UPI / Bank Received (`₹`)** (UPI Sales + UPI Udhaar
Collections).
  - **New Udhaar Given (`₹`)**.
  - **Net Realized Profit (`₹` & Margin %)**: Calculated accurately as
`(Selling Price - Purchase Cost Price)` across all items sold!
- **Top Selling Products & Low-Stock Reorder Sheet**:
  - Table of all items where `stockQty <= lowStockThreshold` with a
1-click **"Print / Copy Distributor Purchase Order List"** button so
the shopkeeper can send their restock list to their city wholesaler on
WhatsApp.

Ensure all 7 navigation views in `App.tsx` are fully connected,
keyboard shortcuts (`Alt+1` through `Alt+7`, `F1`–`F12`) work
globally, and `@media print` styles cleanly isolate thermal receipts,
A4 invoices, and A4 barcode sticker sheets.
```

7. Pitching & Monetization Strategy for Rural Shop Owners (Phase 2 Expansion)

Once you run this live in your own electronics shop for 2–3 weeks, your own billing counter becomes your strongest live demo. Use this exact field strategy to onboard other rural and semi-urban shopkeepers:

1. **Solve the "Zero-Hardware Barrier" First**:
    - Do not ask a rural shopkeeper to buy a ₹15,000 POS machine + ₹12,000 thermal barcode printer on Day 1.
    - Show them how your software runs **offline on any basic laptop** (or second-hand ₹12,000 PC), prints **65 barcode stickers on a ₹1 A4 sticker**

sheet using their regular printer, and uses a **₹900 USB barcode scanner** (or even their camera).

2. **Lead the Pitch with 3 Tangible Pain Killers (Not "Software Features")**:
    - **Pain Killer 1 — "Chori & Bhool Band" (Stop Forgotten Udhaar & Margin Leakage)**: Show how partial Udhaar automatically goes into the village-wise Khata list and sends a 1-click WhatsApp reminder with their UPI ID.
    - **Pain Killer 2 — "Exact Amount UPI QR"**: Show how the laptop screen displays a dynamic QR code where the customer cannot manipulate or mistype the amount.
    - **Pain Killer 3 — "2-Second Warranty & Duplicate Bill Check"**: Show how typing a customer's phone number immediately tells whether a returned bulb, battery, or appliance is still under warranty—saving the shopkeeper thousands of rupees in false warranty replacements.
3. **Rural-Friendly Pricing & Revenue Model**:
    - **Step 1 (Setup + Catalog Onboarding Fee — Immediate Cashflow)**: Charge a one-time **₹1,499 to ₹2,499 "Dukaan Digital Setup Package"** where you (or a local college student agent paid ₹400 commission per shop) install the app on their laptop, enter their first 100 products, and print their first 5 sheets of A4 barcode stickers. Rural shopkeepers willingly pay for *done-for-you setup* far more readily than abstract software licenses.
    - **Step 2 (Hardware Bundle Margin)**: Bundle a plug-and-play USB barcode scanner + 58mm thermal receipt printer + 100 A4 barcode sticker sheets with the software for **₹4,999 all-inclusive** (earning ₹1,500–₹2,000 instant margin on Day 1).
    - **Step 3 (Annual Renewal / Cloud Sync Tier)**: Keep offline laptop billing working reliably, and charge **₹99/month or ₹999/year** (less than ₹3/day — cheaper than a cup of tea) for automatic Cloud Backup, Mobile Owner Dashboard (checking shop sales from home via Supabase), and multi-counter sync.